

Lab 03 — Hands-on lab: life, signals and death of a container

Goal: provoke every behaviour of the course yourself — the immediate stop, the 10 seconds of agony, code 137, automatic restart — and see who, without a daemon, watches over your containers.

Prerequisites — Labs 01 and 02 done. Images `alpine` and `nginx:alpine` present.

Files provided — `files/demarrage-casse.sh` (broken start-up) and `files/demarrage-correct.sh` (correct start-up), used in step 3.

Step 1 — The states, one by one

```
podman create --name state alpine sleep 120
podman ps -a --filter name=state --format 'table {{.Names}}\t{{.Status}}'
```

Observe the status `Created`: the container exists, no process is running.

```
podman start state
podman ps --filter name=state --format '{{.Status}}'
podman pause state && podman ps -a --filter name=state --format '{{.Status}}'
podman unpause state && podman ps --filter name=state --format '{{.Status}}'
podman stop -t 2 state && podman ps -a --filter name=state --format '{{.Status}}'
podman start state && podman ps --filter name=state --format '{{.Status}}'
podman rm -f -t 0 state
```

Observe the succession `Up 1 second`, `Paused`, `Up 5 seconds`, a warning `StopSignal SIGTERM failed to stop container state in 2 seconds, resorting to SIGKILL`, `Exited (137)`, then `Up` again.

Explanation. An `Exited` container is restartable: it kept its configuration and its writable layer. Note that `podman ps` without `-a` **does not show** a paused container: it is not "running".

Step 2 — Why a container stops on its own

```
podman run --name try1 alpine
podman ps -a --filter name=try1 --format '{{.Status}}'
```

Observe `Exited (0)`: `alpine`'s default command is `/bin/sh`, which, with no input, exits immediately.

```
podman run -d --name try2 nginx:alpine
podman ps --filter name=try2 --format '{{.Status}}'
```

Observe `Up`: `nginx` runs in the foreground.

```
podman run --rm alpine sh -c 'sleep 60 & echo "started in the background"'
```

Observe that the command returns **immediately**, even though a `sleep 60` was started.

Explanation. The `&` detached `sleep`; the shell ran `echo` and exited. PID 1 being dead, the container is destroyed, `sleep` with it. That is **the** number-one trap of start-up scripts.

Look at who is watching `try2` while it runs:

```
podman inspect --format '{{.State.CommonPid}}' try2
ps -o pid,ppid,user,comm -p $(podman inspect --format '{{.State.CommonPid}}' try2)
```

Observe a `conmon` process, under **your** user: it is the supervisor Podman leaves behind each container — the only "daemon" you have left, and it weighs only a few hundred KB.

```
podman rm try1 ; podman rm -f -t 0 try2
```

Step 3 — The broken start-up script, and its fix

Copy the two scripts provided:

```
mkdir -p ~/labo-docker/03 && cd ~/labo-docker/03
cp <lab-path>/files/*.sh . && chmod +x *.sh
cat demarrage-casse.sh demarrage-correct.sh
```

Run the first one **inside** a container, mounting the current folder:

```
podman run --rm -v "$PWD":/scripts alpine /scripts/demarrage-casse.sh
```

Observe the message printed, then an immediate return to the prompt: the container is already dead and removed.

```
podman run -d --name correct -v "$PWD":/scripts alpine /scripts/demarrage-correct.sh
podman ps --filter name=correct --format '{{.Status}}'
podman top correct
```

Observe that the container stays `Up`, and that `podman top` shows `sleep 300` as PID 1 — and **no** parent `sh` process.

Explanation. The `exec` of the second script **replaced** the shell with the final command, which inherits PID 1. That is exactly what the `exec` form of an `ENTRYPOINT` does, seen in lab 04.

→ LINUX

`exec` is a shell built-in that calls the system call of the same name: the current process abandons its program (the shell) and loads the requested program **in its place**, keeping its PID. Without `exec`, the shell creates a child (`fork`) and waits. With `exec`, there is no shell any more at all.

```
podman rm -f -t 0 correct
```

Step 4 — The 10 seconds of agony

Measure a stop on a process that ignores `SIGTERM`:

```
podman run -d --name idle alpine sleep 300
time podman stop idle
podman inspect --format 'code={{.State.ExitCode}} oom={{.State.OOMKilled}}' idle
podman rm idle
```

Observe the warning `StopSignal SIGTERM failed to stop container idle in 10 seconds, resorting to SIGKILL`, `real 0m10.1s` and `code=137 oom=false`.

Start again with a mini-init:

```
podman run -d --init --name idle alpine sleep 300
podman exec idle ps -o pid,comm
time podman stop idle
podman inspect --format 'code={{.State.ExitCode}}' idle
podman rm idle
```

Observe `1 podman-init` then `sleep` as PID 2, a stop in `0m0.1s` and `code=143`.

And with an application that handles its signals properly:

```
podman run -d --name web nginx:alpine
time podman stop web
podman inspect --format 'code={{.State.ExitCode}}' web
podman rm web
```

Observe an instant stop and `code=0`.

Explanation. Three behaviours, three causes. `sleep` as PID 1 **ignores** `SIGTERM` (kernel protection): the engine waits then kills → `137`. With `--init`, `sleep` is no longer PID 1, the default action applies → `143`. `nginx` installs a signal handler and terminates cleanly (with code `0`, because `nginx` chose to exit normally). Those ten seconds multiplied by your containers are the unexplained duration of your redeployments.

You can shorten the grace period — without fixing the cause:

```
podman run -d --name idle alpine sleep 300
time podman stop -t 2 idle
podman rm idle
```

Observe `real 0m2.1s`, still with code `137`.

Step 5 — Read exit codes

```
podman run --rm alpine sh -c 'exit 0' ; echo "code=$?"
podman run --rm alpine sh -c 'exit 3' ; echo "code=$?"
podman run --rm alpine missing-command ; echo "code=$?"
```

Observe `0`, `3`, then `127` with `Error: crun: executable file missing-command not found in $PATH`.

```
podman run -d --name killed alpine sleep 300
podman kill killed
podman inspect --format 'code={{.State.ExitCode}}' killed
podman rm killed
```

Observe `137`, immediately this time: `kill` does not wait.

Now provoke a real memory shortage:

```
podman run --name oom --memory=32m --memory-swap=32m alpine sh -c 'head -c 100m /dev/zero |
tail'
echo "code=$?"
podman inspect --format 'code={{.State.ExitCode}} oom={{.State.OOMKilled}}' oom
podman rm oom
```

Observe `code=137` and this time `oom=true`: same code, different cause, and only `inspect` tells the difference.

Explanation. Above 128, the code means death by signal: `code - 128` gives the signal number. `127` on the other hand is a launch error: the application never started.

Step 6 — `exec` versus `attach`

```
podman run -d --name web nginx:alpine
podman exec web nginx -v
podman exec -it web sh
```

In the shell you get, type:

```
ps -o pid,comm
exit
```

Observe that `nginx` is PID 1, followed by its *workers*, and that your `sh` has another PID. When you leave the shell, the container is **still** `Up`.

```
podman ps --filter name=web --format '{{.Status}}'
```

Explanation. `exec` created a **new** process in the container's namespaces. Leaving it does not affect PID 1. `attach`, conversely, would hook you onto `nginx` itself: a `Ctrl+C` would stop it. To read logs, always use:

```
podman logs --tail 5 web
podman logs -f --since 1m web      # Ctrl+C here only stops the display
```

Step 7 — Logs only come from `stdout`

```
podman run --rm --name logs-demo alpine sh -c \
'echo "I go to stdout"; echo "I go to a file" > /tmp/app.log; sleep 1'
```

Observe that only the first line appears.

```
podman run -d --name logs-demo alpine sh -c \
  'echo "visible"; echo "invisible" > /tmp/app.log; sleep 120'
podman logs logs-demo
podman exec logs-demo cat /tmp/app.log
podman rm -f -t 0 logs-demo
```

Observe that `podman logs` prints `visible` and that the file content is only reachable by entering the container.

Explanation. `common` only captures `stdout` and `stderr` of PID 1. That is why a containerised application must log to the console — and why you must not configure `logging.file.name` in a containerised Spring Boot.

Step 8 — Automatic restart

```
podman run -d --restart=on-failure:3 --name unstable alpine \
  sh -c 'echo "start $(date +%T)"; sleep 3; exit 1'
sleep 20
podman ps -a --filter name=unstable --format '{{.Names}} {{.Status}}'
podman inspect --format 'restarts={{.RestartCount}} code={{.State.ExitCode}}' unstable
podman logs unstable
```

Observe a `RestartCount` of `3`, a status `Exited (1)`, and **four** "start" lines in the logs: the initial attempt plus three retries.

Explanation. Logs accumulate from one run to the next on the same container: the first line is the root cause. `.State`, on the other hand, only describes the **last** run.

```
podman rm unstable
podman events --since 2m --until 1s | grep unstable | awk '{print $5, $6}' | uniq -c
```

Observe the event journal: `container start`, `container died`, `container restart` ... It is the only place where you see a container's *history*, not just its state.

Finally check the incompatibility announced in the course:

```
podman run --rm -d --restart=always nginx:alpine
```

Observe Error: the `--rm` option conflicts with `--restart`, when the `restartPolicy` is not "" and "no".

PODMAN

And after a reboot? Test it: `podman run -d --restart=always --name survivor nginx:alpine`, then close **all** your Ubuntu windows and, from PowerShell, `wsl --shutdown`. Reopen Ubuntu: `podman ps` is empty. Nobody re-read the policy — there is no daemon. On a server that role belongs to `systemd` through a Quadlet file (lab 10). On your workstation this is acceptable: your development containers need not survive a reboot. `podman rm -f -t 0 survivor` afterwards.

Step 9 — Extract evidence from a dead container

```
podman run --name autopsy alpine sh -c 'echo "important trace" > /report.txt; exit 2'
podman ps -a --filter name=autopsy --format '{{.Status}}'
podman cp autopsy:/report.txt ./report.txt
cat report.txt
```

Observe that the file is recoverable while the container is `Exited (2)`.

```
podman rm autopsy
podman cp autopsy:/report.txt ./other.txt
```

Observe `Error: container "autopsy" does not exist`: once the container is removed, everything is lost.

Explanation. Operations rule: **inspect before removing**. `cp`, `logs` and `inspect` work on a stopped container, never on a removed one.

Clean-up

```
podman ps -a --format 'table {{.Names}}\t{{.Status}}'
podman rm -f -t 0 idle web correct unstable autopsy try1 try2 killed state oom survivor
2>/dev/null
rm -f ~/labo-docker/03/report.txt
podman ps -a --format '{{.Names}}'
```

Observe that no container from this lab remains. The images `alpine` and `nginx:alpine` are kept.

What you must be able to state now

- A container dies with its PID 1 — you provoked all three cases.
- `sleep` as PID 1 ignores `SIGTERM`; `--init` fixes the symptom, `exec` the cause.
- `137` = killed (by `stop`, `kill` or the OOM killer — `inspect` settles it), `143` = stopped cleanly, `127` = command not found.
- `exec` creates a process, `attach` hooks onto PID 1.
- `podman logs` only shows `stdout` / `stderr`, captured by `common`.
- `podman rm` destroys logs and evidence: inspect first.
- Without a daemon, a *restart policy* does not survive a `wsl --shutdown`.